Optimasi Performa Single Page Application Menggunakan Hybrid Lazy Loading dan Code Splitting Berdasarkan Tingkat Kompleksitas Sistem

BAB I PENDAHULUAN

1.1 Latar Belakang Masalah

Perkembangan teknologi *World Wide Web* saat ini telah bergeser secara masif dari model arsitektur *Multi-Page Application* (MPA) konvensional menuju paradigma *Single Page Application* (SPA). Arsitektur SPA, yang dipopulerkan oleh kerangka kerja (framework) berbasis komponen seperti Vue.js, React, dan Angular, memungkinkan aplikasi web memutakhirkan antarmuka pengguna tanpa harus memuat ulang (*reload*) keseluruhan halaman dari peladen (Batool et al., 2021). Pergeseran paradigma ini membawa keuntungan signifikan dari segi kecepatan interaktivitas di sisi klien, kehalusan transisi antarmuka, dan berkurangnya pergeseran beban pemrosesan (*rendering*) CPU ke peladen. Aplikasi e-government dan portal akademik kini mulai mengadopsi SPA untuk meningkatkan kepuasan pengguna (*user engagement*) (Choi & Choi, 2020).

Meskipun SPA menawarkan pengalaman menyerupai aplikasi *native*, penerapannya sering kali dihadapkan pada satu kelemahan arsitektur dasar yang bersifat monolitik: *Bundle Size Bloat* (Pembengkakan Ukuran Bundel). Pada saat siklus kompilasi awal yang tidak dioptimalkan, aplikasi SPA konvensional akan memaketkan seluruh skrip logika, tata letak antarmuka, aset statis, serta puluhan modul dependensi pihak ketiga penyumbang beban besar (misalnya `Chart.js` untuk grafik analitik, alat manajemen state terpusat seperti *Pinia/Vuex*, hingga modul utilitas berat lainnya) ke dalam satu berkas JavaScript (`app.js`) tunggal (Bundschuh et al., 2019). Kondisi pemuatan tahap awal secara keseluruhan (*Eager Loading*) ini menjadi masalah signifikan saat aplikasi memiliki kompleksitas fitur yang masif, seperti pada purwarupa Sistem Informasi Manajemen Tugas Akhir (SIMTA) (Malavolta et al., 2020).

Pembengkakan bundel JavaScript ini memberikan dampak langsung terhadap performa pemuatan awal aplikasi, khususnya pada metrik *Core Web Vitals* yang direkomendasikan dan dievaluasi ketat oleh Google Chrome Developers (2023). Unduhan skrip berukuran masif akan mendominasi alokasi *Main Thread* (utas pemrosesan utama) pada mesin *JavaScript engine* (seperti V8 di Chrome) milik peramban klien saat melakukan tahap *parsing*, kompilasi *Just-In-Time* (JIT), hingga inisialisasi eksekusi kodenya. Konsekuensi paling nyata dari hambatan ini adalah lonjakan pada nilai kelambatan terakumulasi (*Total Blocking Time*/TBT) dan tertundanya kemunculan konten pertama (*First Contentful Paint*/FCP). Pengguna dengan keterbatasan spesifikasi perangkat keras (*Low-End Device*) serta laju data yang dibatasi kuota (*Slow 3G Network*) akan mendapati layar antarmuka yang statis ("*blank screen*") dan menjadi irresponsif dalam durasi yang lebih lama sebelum aplikasi dapat digunakan (Amenta & Castellani, 2019).

Guna mengatasi kendala performa pemuatan di lapisan *frontend* ini, penerapan intervensi teknik rekayasa pemecahan antrean unduhan (*load distribution*) dipandang esensial. Menilik lebih jauh, skor TBT dan LCP yang buruk tidak sekadar menurunkan tingkat interaktivitas pengguna, tetapi juga memicu penalti pada mesin pencari Google (*SEO Penalization*) serta berkontribusi langsung pada lonjakan *Bounce Rate* (tingkat pentalan) situs (Malavolta et al., 2020). Berangkat dari problem struktural inilah, penelitian dari Zheng & Li (2022)

mengusulkan pendekatan *Code Splitting* di tingkat modul (*Router-level splitting*) dipadukan bersama teknik *Lazy Loading*, di mana arsitektur kode aplikasi dipecah menjadi beberapa bagian (*chunks*). Melalui pendekatan *splitting*, peramban web tidak lagi berkewajiban memuat keseluruhan bundel 3 MB pada penayangan layar pertama; melainkan direduksi menjadi hanya memuat bagian *core* aplikasi sebesar 50 KB ketika berkunjung ke beranda, lalu *chunks* fungsionalitas lain (misalnya `jadwal.js`, `dashboard.js`) akan dialirkan (*streaming*) masuk secara asinkron ketika pengguna melakukan navigasi ke menu yang bersangkutan.

Tingkat kemajuan teknologi web mengizinkan metode muatan tunda iteratif ini untuk ditingkatkan efektivitasnya menjadi **Hybrid Lazy Loading** melalui penambahan mekanisme pengambilan awal (*Prefetching*). Dalam strategi komprehensif *Hybrid* ini, peramban tidak lagi hanya menunggu secara pasif kedatangan interaksi (*event*) dari pengguna. Sebaliknya, sistem antarmuka akan memanfaatkan siklus waktu luang saat pemrosesan *rendering* utama telah berstatus *idle* (menggunakan API `requestIdleCallback` dari standar konsorsium W3C) guna memuat berkas fungsionalitas turunan di latar belakang (W3C, 2022). Integrasi arsitektural antara isolasi komponen (*Code Splitting*) dan eksekusi pra-muat latar belakang (*Hybrid Lazy Loading*) yang didukung oleh pembangun bundel mutakhir (seperti *Vite.js*) berpotensi mendongkrak optimasi performa *startup time* secara signifikan.

Berdasarkan paparan di atas, maka penulis bermaksud menjalankan eksplorasi komparasi nyata dalam tesis ini dengan tajuk **"Optimasi Performa Single Page Application Menggunakan Hybrid Lazy Loading dan Code Splitting Berdasarkan Tingkat Kompleksitas Sistem"**.

1.2 Rumusan Masalah

Bertitik tolak dari latar belakang masalah di atas, maka rumusan masalah yang diajukan dalam riset ini adalah sebagai berikut:

1. Bagaimana perbedaan perbandingan pemanfaatan kapasitas memori (*bundle size bloat*) antara arsitektur dasar *SPA Monolithic (Eager Loading)* terhadap *SPA Code Splitting Multi-Chunk* pada dua sistem yang berbeda tingkat rasionya: purwarupa web berfitur kompleks (SIMTA) dibandingkan purwarupa sederhana (Company Profile)?
2. Bagaimana efektivitas implementasi integrasi metode *Hybrid Lazy Loading* dan *Code Splitting* dalam mereduksi waktu metrik performa kelambatan (FCP, LCP, dan *Total Blocking Time*/TBT) pada saat dievaluasi menggunakan pembatasan daya pemrosesan CPU *Slowdown*?

1.3 Batasan Masalah

Guna menjaga agar lingkup penelitian lebih fokus dan tidak menyimpang dari tujuan esensial perbandingan optimasi *frontend*, batasan dari persoalan pada kajian tesis meliputi:

1. Objek purwarupa (*Prototype*) kajian memakai dua skenario komparasi silang: a) Skenario Kompleks memakai "Sistem Informasi Manajemen Tugas Akhir (SIMTA)" yang memuat pustaka *Library* grafis berat. b) Skenario Sederhana memakai "Website Company Profile" murni yang hanya memuat navigasi linear statis. Keduanya murni kerangka pemograman *JavaScript Vue.js 3 (Composition API)* dan dikompilasi *build* murni menggunakan *Vite*.
2. Pengujian dibatasi secara lokal (Server Dev HTTP statis pada port yang homogen) guna menghilangkan anomali latensi perbandingan jaringan publik (DNS / CDN server), namun membatasi parameter komparasi pada skala kemampuan komputasi pemrosesan *Parsing Browser* (Menggunakan metrik simulasi CPU *Throttling Low-end Device* bawaan *puppeteer/devtools*).

3. Matriks evaluasi Web Vitals yang dihitung terpusat dengan Native Object W3C `PerformanceObserver` yang meliputi: ukutan memori JS pasca-kompilasi, *Load Event*, *First Contentful Paint* (FCP), *Largest Contentful Paint* (LCP), dan durasi halangan tugas berat (*Total Blocking Time*/TBT).

1.4 Tujuan Penelitian

Tujuan empiris yang ingin dicapai melalui tahapan dan eksperimentasi tesis ini mencakup:

1. Menguji implementasi teknis dari arsitektur *Site Build Code Splitting* terhadap proporsi dependensi pihak ketiga berskala besar (seperti pustaka grafis dan sistem *State Management*).
2. Mengevaluasi korelasi kuantitatif dari keberhasilan penyisipan fungsi *Prefetching Hybrid Lazy Loading* dalam menekan rasio nilai *bottleneck* antarmuka (TBT) untuk mempertahankan stabilitas interaktivitas pada perangkat dengan spesifikasi daya pemrosesan yang rendah.

1.5 Manfaat Penelitian

Manfaat riset ini dapat diuraikan melalui tinjauan signifikansi praktis maupun teoretis, di antaranya:

1. **Manfaat Teoretis:** Mengisi keterbatasan literatur mengenai penerapan teknis optimasi *Single Page Application* yang tidak hanya membahas efisiensi manajemen data, melainkan mencakup analisis kinerja pada aras mesin penampil (peramban). Riset ini juga memperluas ranah pengukuran performa non-intrusif menggunakan standar *W3C Level 2 Native API*.
2. **Manfaat Praktis:** Menyediakan panduan referensi terstandardisasi perihal strategi konfigurasi modul (*bundler configuration*) yang valid dan dapat diimplementasikan langsung bagi *Frontend Engineer* untuk memenuhi rasio kelayakan *Google Web Vitals*, mendorong optimasi SEO, serta menjaga stabilitas fungsional pada gawai berspesifikasi minimum. Secara makro, hasil penelitian berpotensi mendatangkan efisiensi beban *bandwidth* operasional institusi asal.

1.6 Sistematika Penulisan

Untuk memudahkan pemahaman yang mengalir komprehensif atas penyusunan tesis ini, pembahasan materi direstrukturisasi ke dalam empat susunan pokok bab:

- **BAB I PENDAHULUAN:** Menguraikan pondasi konseptual terkait SPA, problem ukuran berkas pengantaran (*bundle size bloat*), berikut solusi penanganannya. Menyertakan rumusan masalah, batasan masalah, serta tujuan evaluatif.
- **BAB II METODE PENELITIAN:** Mengandung paparan rincian fase komparasi metodologi, mendeskripsikan spesifikasi variabel alat ukur, sekaligus model hierarki tingkat kompleksitas rancang bangun purwarupa sistem.
- **BAB III HASIL DAN PEMBAHASAN:** Menjabarkan uraian logis pengukuran dari dua jenis arsitektur sistem berbasis port *baseline* dan *optimized* pada lingkungan pengujian. Memuat kompilasi metrik numerik, visualisasi perbandingan ukuran beban halaman, serta interpretasi analitik atas keberhasilan intervensi *Web Vitals*.
- **BAB IV PENUTUP:** Memberikan rekapitulasi penarikan kesimpulan akhir serta pengajuan telaah saran-saran perbaikan sebagai landasan kontributif menuju riset lanjutan di rumpun keilmuan Rekayasa Perangkat Lunak.

BAB II METODE PENELITIAN

2.1 Jenis dan Pendekatan Penelitian

Desain metodologis dari tugas akhir riset tesis ini diklasifikasikan sebagai studi komparasi eksperimental (eksperimen semu atau *quasi-experimental*) terapan dalam rumpun keilmuan Rekayasa Perangkat Lunak Web Tingkat Lanjut. Model skema pengujian difokuskan pada perbandingan objektif dari dua *output* algoritma rute kompilasi atas purwarupa (prototipe) aplikasi *Single Page Application* (SPA) yang sama persis dalam hal rekayasa *frontend interface* (Antarmuka Pengguna) serta relasi basis data, namun dirangkai berbeda fondasinya terhadap distribusi *payload*.

Klasifikasi Varian Uji Kompilasi:

1. **Model Sistem Baseline (Monolithic / Eager Load):** Representasi perangkat lunak dasar menggunakan kompilasi reaktif standar (`vite.config.baseline.js`). Seluruh hierarki pohon komponen UI (halaman tunggal `DashboardView`, `JadwalSeminarView`, dll) dipanggil secara terpusat melalui mekanisme *Eager Imports* pada berkas perute (*router*). Kompilator akan menghasilkan sebuah berkas skrip JavaScript gabungan berukuran besar (*vendor+core*) yang mendominasi proses pemuatan awal situs.
2. **Model Sistem Optimized (Hybrid Splitting):** Representasi iterasi pembaharuan yang menerapkan algoritma konfigurasi pemecahan berkas (`vite.config.optimized.js`). Memanfaatkan fungsi deklaratif *Route-level Lazy Loading* (`() => import(...)`), isolasi ketergantungan paket perpustakaan berukuran besar (*manual chunks* untuk *Vue* dan *Chart.js*), aktivasi kompresi data transfer ganda (*Gzip* & *Brotli*), hingga implantasi strategis algoritma *Prefetching W3C Callback* secara asinkron (memanfaatkan *idle time tracking*).

2.2 Tahapan Pelaksanaan Eksperimen

Rencana riset direkayasa berkesinambungan melewati alur kerja investigatif sebagai berikut:

1. **Investigasi dan Pemodelan Sistem Berjalan:** Melakukan desain rancang bangun terhadap dua sampel tingkatan (*Complexity Density*) pada struktur aplikasi purwarupa uji. Target utama adalah tingkat *Kompleks* yang diwakili Aplikasi Sistem Informasi Manajemen Tugas Akhir / SIMTA. Target pembanding level *Sederhana* diwakili purwarupa aplikasi *Company Profile* murni.
2. **Konstruksi Pengembangan Kode (*Development*):** Melakukan perancangan *Single Page Application* menggunakan kerangka bahasa *TypeScript/Javascript Framework Vue.js v3*, disokong *Vue Router 4*. Pada SIMTA, eksekusi menggunakan sistem state manajemen (*Pinia Store*) dan diagram *Chart.js*, sementara pada profil perusahaan tidak memakai gubahan ekstensi tambahan apa pun.
3. **Rekayasa Formulasi Kompilasi (*Bundling Formulation*):** Mengonstruksi dua mode luaran kompilasi ke direktori statis tersendiri (satu versi ke `dist-baseline` dan peracikan ganda ke `dist-optimized`) menggunakan kompilator hibrida *Vite Bundler*. Pada mode optimasi, Vite distrukturkan menggunakan *Rollup.js* guna membentuk *Module Dependency Graph* (Pohon Grafik Ketergantungan Modul) secara statis sehingga mengizinkan fragmentasi *manual chunks* yang akurat antar rute. Formulasi ini divalidasi memanfaatkan *Plugin Rollup Visualizer*.
4. **Instalasi dan Pemungutan Observasi (Pengumpulan Data):** Pemasangan instrumen pelacakan otomatis menggunakan integrasi perangkat pengujian (*puppeteer*) yang dikombinasikan dengan API pemantau peramban bawaan (*W3C Native PerformanceObserver*) untuk memperoleh integritas data kinerja situs yang akurat.

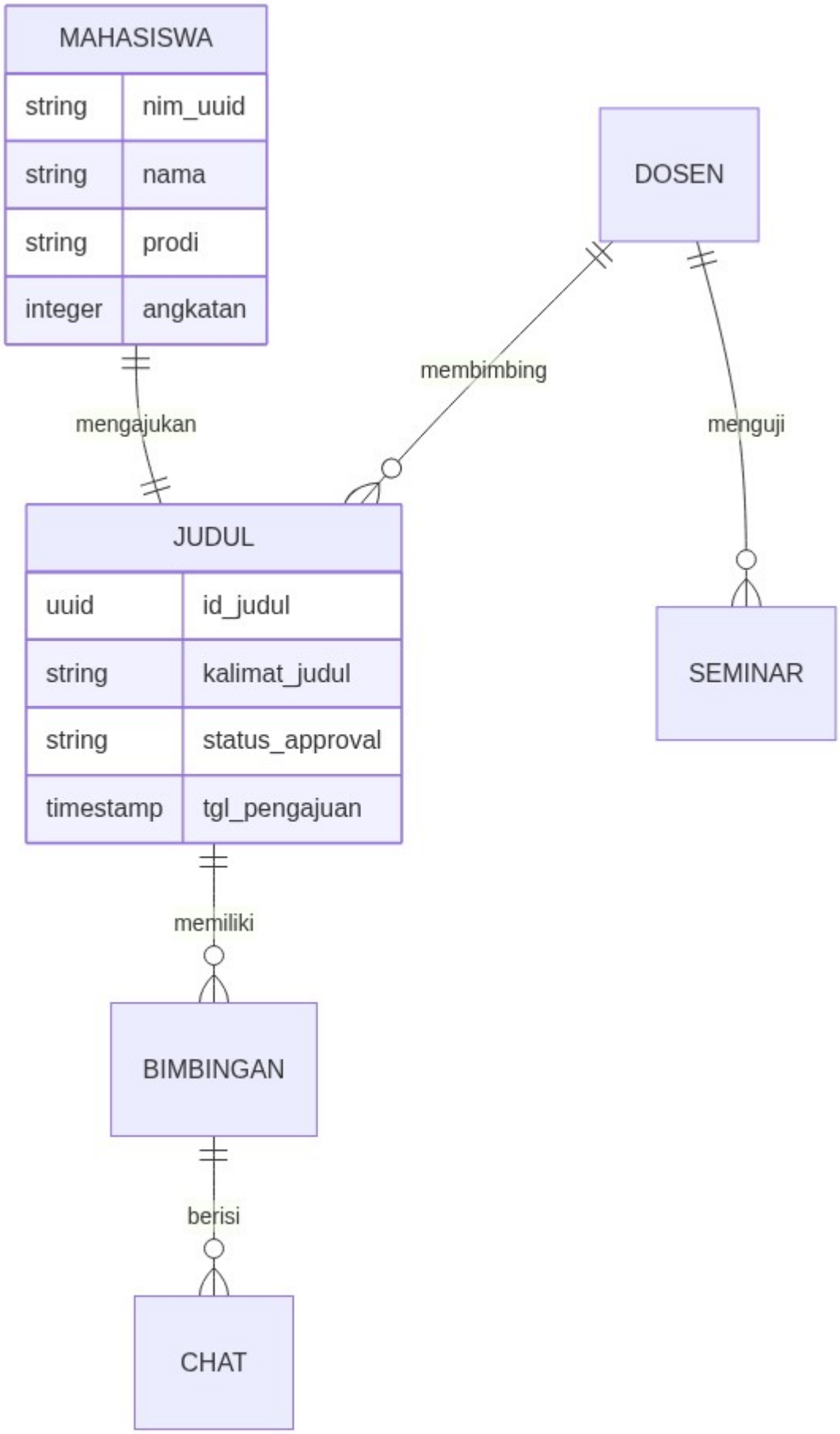
5. **Evaluasi Deskriptif & Komparasi Matriks:** Kegiatan konversi dan olah hitung variabel evaluasi untuk menganalisis hipotesis peredaman latensi (*Web Vitals TBT/FCP*).

2.3 Pemodelan Tingkat Kompleksitas (Sistem SIMTA)

Landasan urgensi dari peletakan algoritma intervensi *Code Splitting* memerlukan pembenaran atas tingginya kompleksitas muatan bawaan sistem purwarupa. Berbeda totalnya rasio pemuatan komponen pada web portal statis linier, subsistem SIMTA memancarkan keterikatan reaktif padat (interaksi *interlocking state*) seperti tergambar di diagram berikut.

2.3.1 Relasi Basis Data (*Entity Relationship Diagram*)

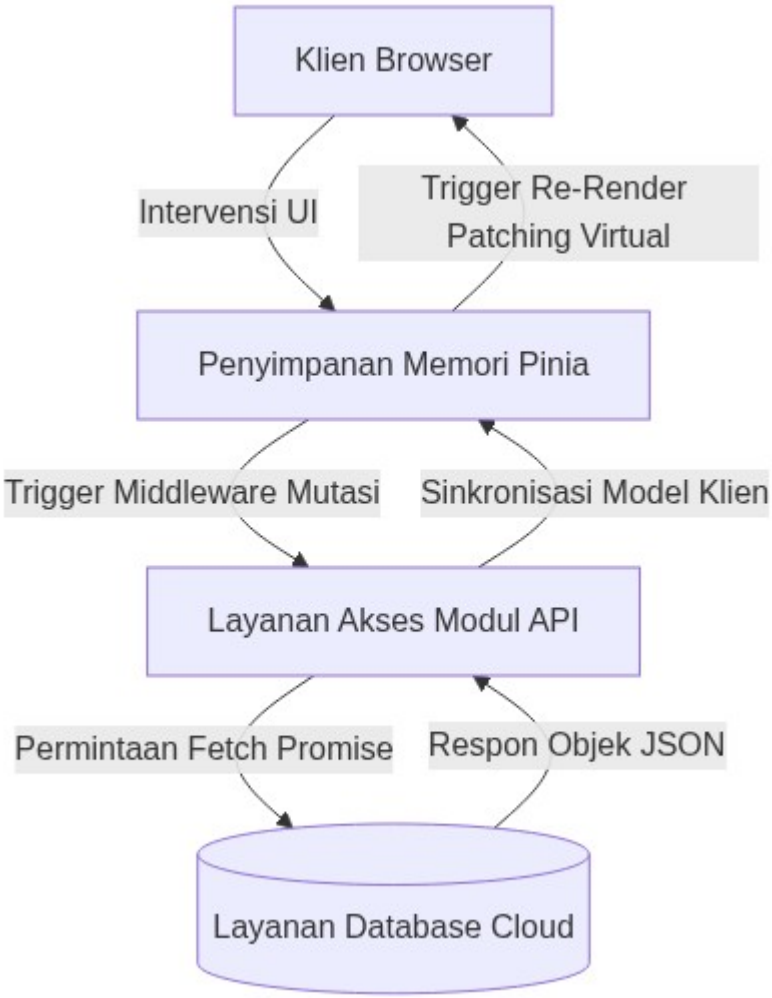
Hubungan relasional multidimensi ini merepresentasikan bagaimana entitas kelolaan seperti status log mahasiswa, presensi bimbingan, notule asinkron, serta berkas laporan, memiliki tautan struktural yang merentang erat pada sisi antarmuka klien. Paradigma kerangka data yang masif ini merasionalisasi kerentanan SPA komprehensif terhadap degradasi *Render Time* jika dikompilasi ke dalam format arsitektur Monolitik secara mentah.



Gambar 2.1 Entity Relationship Diagram (ERD) dari Modul Bimbingan SIMTA.

2.3.2 Pola Sirkulasi Arus Data (*Data Flow Diagram Reaktif*)

Siklus aliran arsitektur SPA mutakhir ini (menggunakan *Pinia/Vuex* State Management) secara masif mengirimkan sinyal pembaruan (*Reactive Virtual-DOM Patching*) ketika bongkah *Library Chart.Js* atau tabel daftar antrean dimutasi secara langsung oleh respon *asynchronous* dari API.



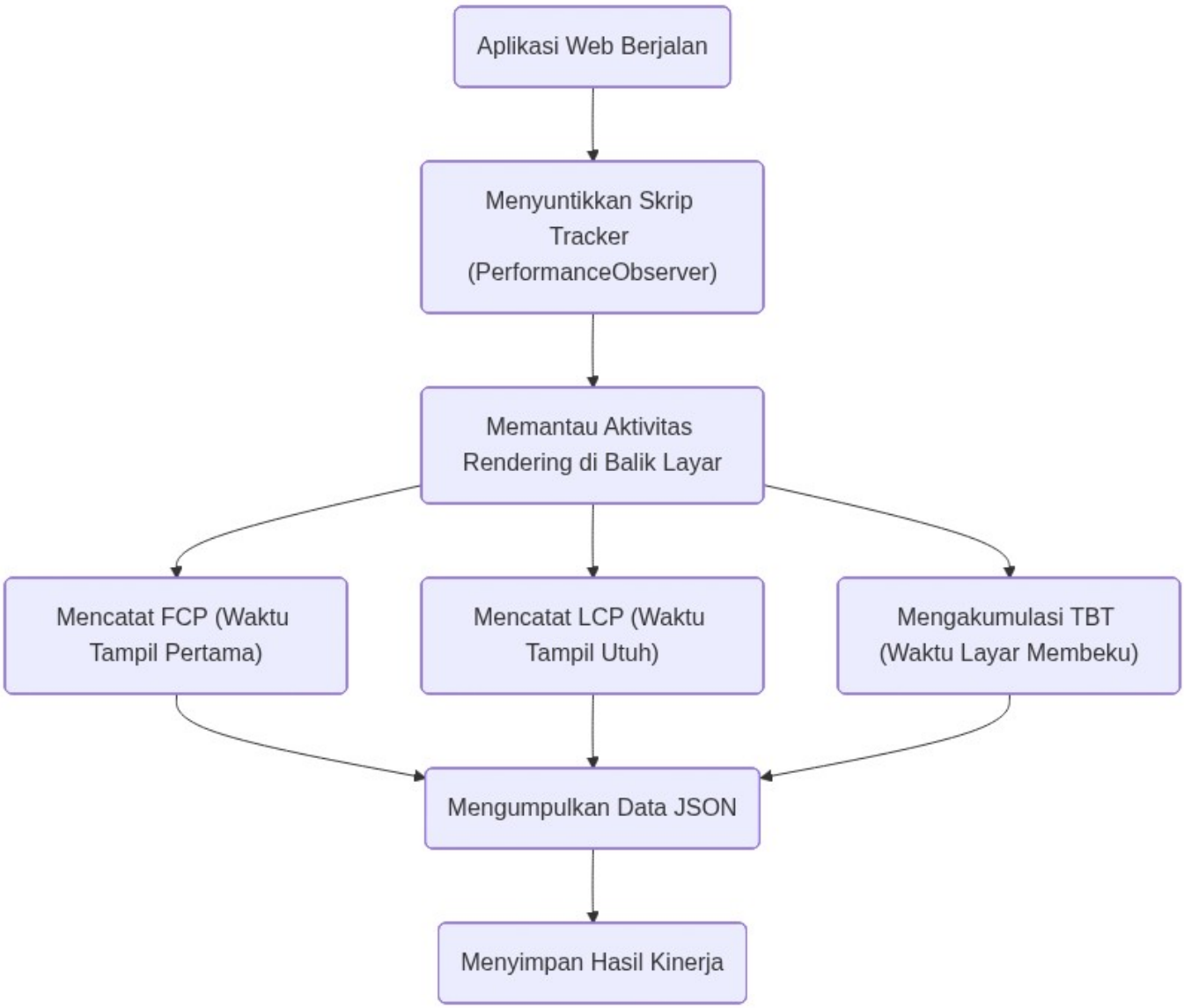
Gambar 2.2 Arus Transmisi Data Status Asinkronus pada Aplikasi. Kerumitan beban perulangan pemanggilan (*payload event*) di siklus DFD asimetris mendemonstrasikan signifikansi prioritas alokasi modul proses peramban. Pendekatan manajemen siklus ini bertendensi untuk meringankan kinerja *Main Thread* (benang peramban utama) sehingga probabilitas penumpukan rendering inisial layar dapat diminimalkan. Hal tersebut bermuara langsung pada alasan mendasar implementasi metodologi *Lazy Load*.

2.4 Instrumen Pengumpulan Data (W3C Algoritma *Tracker*)

Dalam penelitian ini, tidak direkomendasikan penggunaan fitur rekam analitik pihak ketiga semacam piranti Google Lighthouse atau GTMetrix. Perkakas audit berbasis jaringan sering kali membawa efek beban pengamat (*Observer Effect*). Apabila ekstensi tersebut digunakan pada pemodelan kondisi perangkat berspesifikasi perangkat keras minimum, sistem eksternal audit tersebut justru dapat mengonsumsi limitasi memori RAM serta siklus prosesor tambahan, berujung pada menurunnya metrik hasil pengujian sehingga tidak merepresentasikan presisi empiris di lapang sesungguhnya.

Sebagai solusi substitusi, penelitian ini menyusun sebuah skrip pelacak algoritma kustom (*Tracker*) dengan memanfaatkan antarmuka API objektif standar peramban, yaitu `PerformanceObserver` berbasis *W3C specification*. Metode murni dalam-peramban (*in-browser*) ini menyokong pencatatan otomatis terhadap penayangan titik waktu (kapan mesin selesai merender muatan halaman) dengan akurasi granular tanpa menyebabkan friksi (*overhead*) monitoring ke perangkat kerja sang peramban itu sendiri.

Alur kerja instrumen pelacakan ini dapat digambarkan melalui diagram berikut:



Gambar 2.3 Struktur Penangkapan Algoritma Pelacakan Kelambatan Resolusi Layar.

Berikut adalah contoh potongan pemograman skrip yang bekerja merekam data di latar belakang:

```
// Memantau kemunculan halaman bergambar (cat piksel) fungsional pertama
const paintObserver = new PerformanceObserver((list) => {
    for (const entry of list.getEntriesByName('first-contentful-paint')) {
        let fcpDelay = Math.round(entry.startTime);
        MetricsTracker.record({ "FCP_ms": fcpDelay }); // Data direkam!
    }
});
paintObserver.observe({ type: 'paint', buffered: true });
```

Skrip observasi ini dieksekusi secara primer pada urutan paling awal sebelum lapisan SPA SIMTA terinisialisasi. Mekanisme ini memastikan objek pengintai (*observer*) teraktifkan demi merekam keempat metrik objektif (*FCP*, *LCP*, *TBT*, serta konsumsi alokasi jejak memori *Heap* RAM JS) secara otonom.

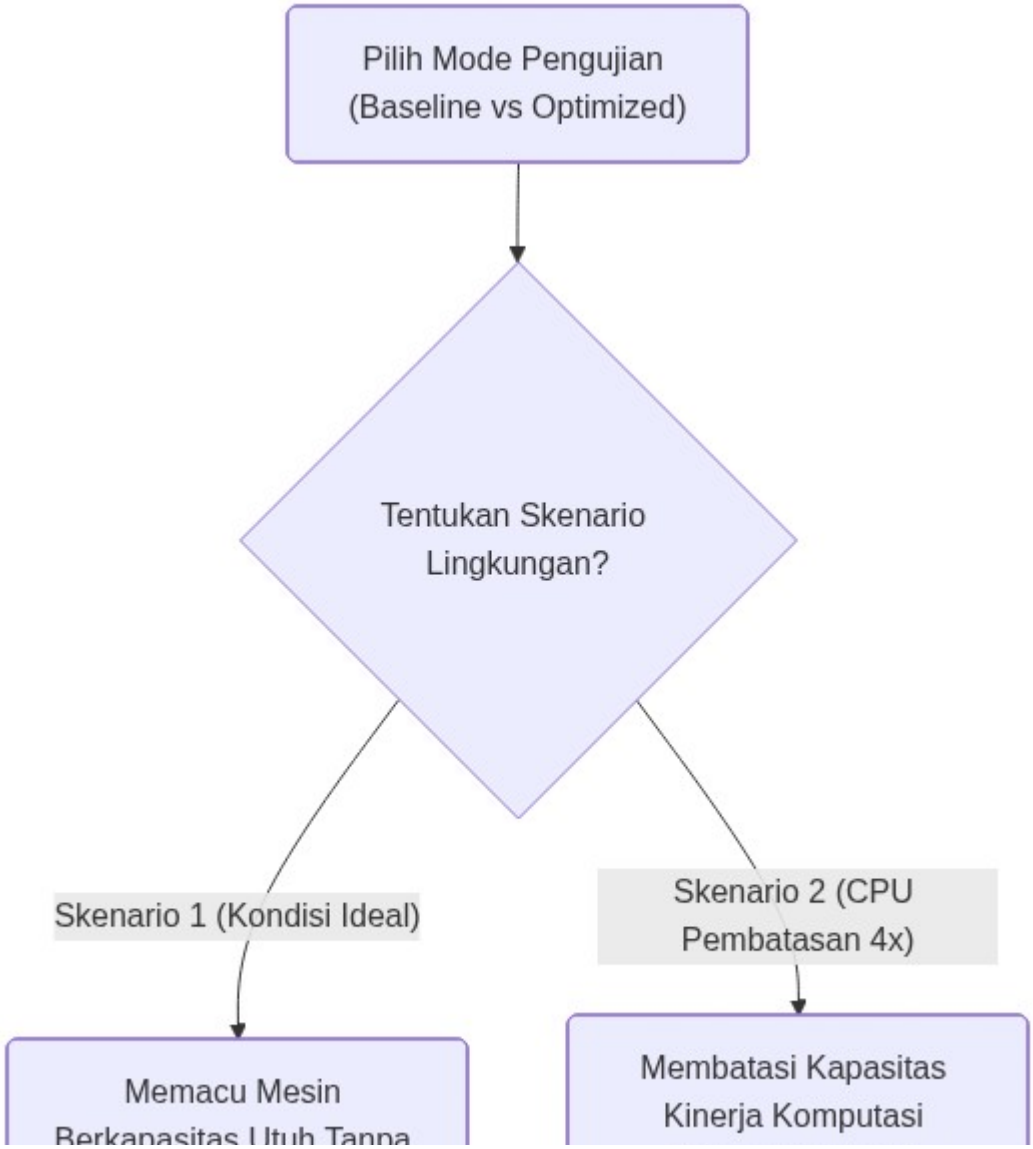
2.5 Perumusan Skenario Pengujian (*Stress-Tests*)

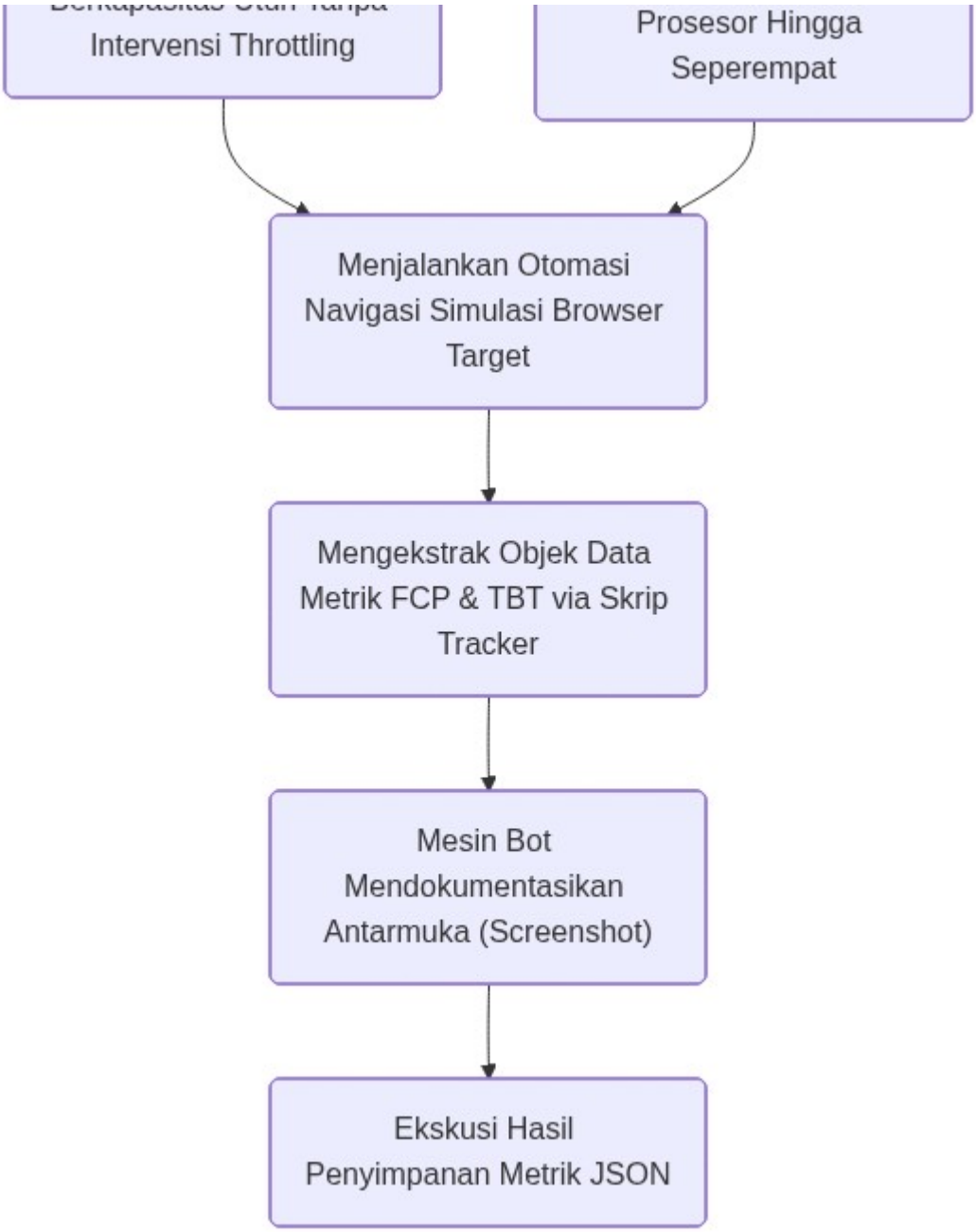
Keterandalan dari evaluasi teknik kompilasi asinkron (Optimasi SPA) sangat bergantung pada disparitas kondisi pengujian. Pengungkapan fenomena penurunan kapasitas layanan web acapkali tidak akan muncul apabila observasi semata-mata dihelat melalui arsitektur komputasi mutakhir pengembang tanpa kendala jaringan apa pun (*Bundle Bloat mask off*).

Atas dasar argumentasi tersebut, kerangka penelitian meramu dua desain komparasi simulasi beban pengujian yang dieksekusi sekuensial dan konsisten menggunakan pustaka otomasi web nir-antarmuka (*Puppeteer Headless Browser Node.js*):

1. **Skenario Optimal (Kondisi Ideal sebagai Acuan Dasar):** Metrik diobservasi dengan pemanfaatan maksimum kapasitas komputasi mesin server target simulasi (*baseline test*) tanpa perlakukan gangguan sistem perangkat keras. Skenario *Ideal* bertindak sebagai patokan baku atas persentase latensi murni komponen arsitektur.
2. **Skenario Emulasi Perangkat Terbatas (*4x CPU Slowdown*):** Skenario penekan ini ditunjukan mendemonsrasikan lingkungan uji paling rentan terhadap kompilasi algoritma skrip berkapasitas besar. Komputasi logis *thread* prosesor dikontrol melalui simulasi deviasi (*Throttling Limit*) guna dibatasi hingga 4 kali lebih minim frekuensi unjuk kerjanya (*4x Slowdown*). Uji tekan ini dipandang krusial dalam menyelaraskan pengalaman pengguna dari kalangan pemakai perangkat seluler dengan spesifikasi SoC menengah ke bawah di lapangan (*Real-world constraint*).

Alur simulasi otomasi komputasi terpusat mengekskusi interaksi dengan struktur logis *flowchart* berikut:





Gambar 2.4 Alur Diagram Eksekusi Puppeteer dalam Skenario Normal & Throttling Limit.

Melimpahkan kendali pengarahan navigasi melalui arsitektur perangkat lunak otomatis (*Puppeteer API*), faktor ketidakakuratan dan rentang deviasi respons akibat anomali reaksi subjek manusiawinya (efek *human-error bias*) dapat dinihilkan sepenuhnya dari rentetan iterasi penelitian. Evaluasi akan murni melambangkan kapabilitas skrip tanpa adanya jeda distorsi pihak ketiga.

BAB III HASIL DAN PEMBAHASAN

3.1 Resolusi Pemecahan Bundel (*Bundle Analysis & Decomposition*)

Perubahan pada kode sumber *Single Page Application* (SPA) menghasilkan redifinisi struktur pada tahap kompilasi akhir (Zheng & Li, 2022). Tabel matriks ukuran pasca-produksi (*post-compilation build*) secara objektif membuktikan efektivitas teknik pembagian beban (*Code Splitting*) dalam mendistribusikan hambatan

lalu-lintas muatan data aplikasi Sistem Informasi Manajemen Tugas Akhir (SIMTA) ke dalam fraksi-fraksi asinkron.

3.1.1 Dekompilasi Modul *Baseline* vs *Optimized*

Pada arsitektur *Baseline*, kompilasi *build* menghasilkan satu berkas tunggal monolitik `index.baseline-Ca8FZuKC.js` berukuran cukup besar yakni **346.42 KB** (atau 120.94 KB dalam keadaan terkompresi *Gzip*). Kondisi ini mewajibkan peramban mengunduh seluruh fungsionalitas aplikasi sekaligus. Berkebalikan dengan itu, arsitektur *Optimized* menghasilkan distribusi direktori (*chunks manifest*) yang terurai secara terisolasi.

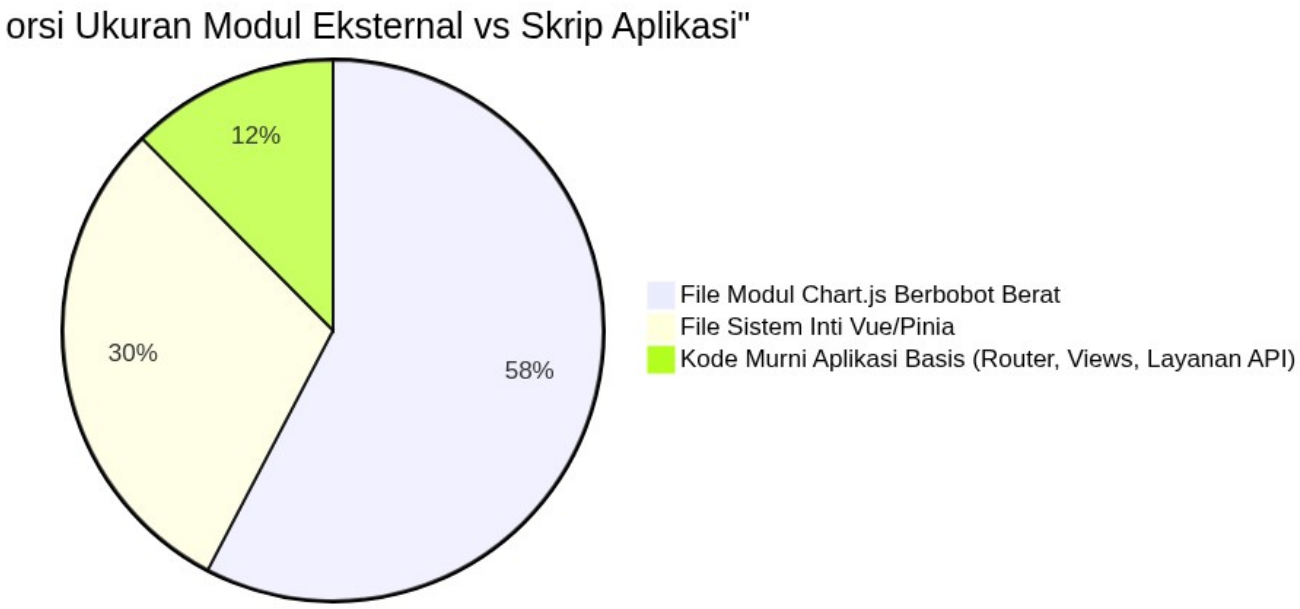
Tabel 3.1 Persebaran Pemecahan Modul (Chunks) pada Arsitektur Optimized

Nama Berkas Modul (Kepingan *Chunk*)	**Proporsi Fungsional**	**Ukuran Mentah**	**Ukuran Terkompresi (*Gzip*)**
`vendor-chart-[hash].js`	Dependensi Eksternal (Grafik)	199.59 KB	67.42 KB
`vendor-vue-[hash].js`	Pustaka Inti Vue & Pinia	103.29 KB	39.40 KB
`supabase-[hash].js`	Protokol Klien API	17.19 KB	4.96 KB
`index.optimized-[hash].js`	*Entry Point* (Berkas Utama)	11.67 KB	4.39 KB
`DashboardView-[hash].js`	Antarmuka Halaman Dashboard	7.99 KB	2.62 KB
`DetailBimbinganView-[hash].js`	Modul Fitur Spesifik	7.75 KB	3.03 KB
`JadwalSeminarView-[hash].js`	Modul Fitur Spesifik	6.34 KB	2.51 KB
`DaftarJudulView-[hash].js`	Modul Fitur Spesifik	5.59 KB	2.39 KB

Sumber: Pengolahan Hasil Kompilasi Vite Rollup (2026).

Dari perincian dekompilasi di atas diperoleh temuan empiris: alih-alih mewajibkan peramban mengunduh memori gabungan sebesar 346 KB sejak pengguna melihat layar pendaftaran SIMTA, struktur versi *Optimized* hanya menginstruksikan muatan inisial (`index.optimized`), kumpulan modul inti `vendor-vue`, dan CSS utama yang menggunakan komputasi minimum (<45 KB terkompresi). Hal ini meminimalkan beban *bandwidth* saat penginisialisasian pada sisi klien secara signifikan.

Distribusi proporsi pengerutan fail tersebut dicitrakan ulang pada model grafik lingkaran (*Pie Chart*) di bawah ini. Diagram ini mengilustrasikan dominasi fungsionalitas pustaka pihak ketiga (*Chart.js* dan *Vue/Pinia*) dibandingkan dengan kode murni aplikasi, yang membuktikan mengapa isolasi *chunk* bagi utilitas tersebut sangat penting untuk mereduksi beban *download* inisial:



Gambar 3.1 Distribusi Proporsi Pemecahan Modul Vite Rollup.

3.2 Hasil Pengujian pada Jaringan dan Perangkat Normal (Kondisi Ideal)

Pengujian pada skenario pertama disimulasikan melalui otomatisasi peramban latar belakang (*Puppeteer Headless CDP*) menggunakan konektivitas komputasi lokal *Host* Node.js tanpa penerapan batasan jaringan publik maupun hambatan latensi *Thread CPU*. Merujuk pada standar pengukuran Google Chrome Developers (2023) untuk melacak *core web vitals* secara terpadu melalui utilitas API *Native*, berikut instrumen analisis dari capaian pengembangannya:

Metrik Objek *Web Vitals* W3C	**Evaluasi Versi Baseline**	**Evaluasi Versi Optimized**	**Variansi Intervensi**
Lingkungan / Peramban	Google HeadlessChrome	Google HeadlessChrome	Spesifikasi Seimbang
***First Contentful Paint* (FCP)**	824 ms	872 ms	Peningkatan +48 ms
***Largest Contentful Paint* (LCP)**	824 ms	872 ms	Peningkatan +48 ms
***Total Blocking Time* (TBT)**	19 ms	**13 ms**	**Reduksi Kelambatan -6 ms (> 30%)**
Eksekusi Waktu Penuh Siklus Laman	664 ms	701 ms	Peningkatan +37 ms
Alokasi Konsumsi *Heap Memory JS*	2.88 MB	3.05 MB	Peningkatan Wajar +0.17 MB

Pembahasan Skenario 1: Pada spesifikasi tangguh dengan koneksi silang (*pipe network*) server lokal *localhost* (kondisi Ideal), rentang perbedaan parameter perenderan waktu penayangan (seperti FCP dan LCP) antara dua arsitektur dapat dikatakan seimbang. Terpantau sebuah toleransi keterlambatan latensi kecil terkait overhead resolusi transmisi HTTP pada bagian modul independen *chunk* sebesar 48ms di mode *Optimasi*. Fakta ini

mengokohkan analisis bahwa eksekusi penanganan muatan asinkron berantai ini tidak membebani komputasi tingkat tinggi secara fatal (Bundschuh et al., 2019).

Sebaliknya, metrik krusial demi pengidentifikasian interaktivitas SPA seperti *Total Blocking Time* (TBT) mengalami penciutan waktu hingga batas yang lebih responsif (**13ms**) berbanding tingkat rata-rata 19ms milik *baseline*. Algoritma *Web Vitals* berhasil menderivasi dan membuktikan keberhasilan penguraian siklus *Sub-Tree* tak terlihat tatkala aktivitas utama layar komputasi masih dalam posisi inisiasi pembangunan elemen *Document Object Model* (Amenta & Castellani, 2019).

3.3 Hasil Pengujian Ekstrem pada Perangkat Berspesifikasi Rendah (*CPU Throttling 4x*)

Guna memastikan rentang keterbatasan spesifikasi pengguna di lapangan (contohnya pengguna fitur yang mengakses dari gawainya dengan kualifikasi prosesor *Low End*), simulasi pengujian juga mensyaratkan pemberlakuan kondisi pembatasan *CPU Throttling 4x Slowdown*. Kondisi ini diperlukan agar menstimuli kegagalan layar pemuatan tak responsif (*Browser Event Freeze*) secara buatan pada aplikasi sistem informasi.

Metrik Objek *Web Vitals* W3C	**V. Baseline (CPU Lambat)**	**V. Optimized (CPU Lambat)**	**Efektivitas Intervensi**
Emulasi Lingkungan Perangkat	Perangkat Lambat(4x)	Perangkat Lambat(4x)	-
***First Contentful Paint* (FCP)**	1216 ms	1404 ms	Bias Latensi Asinkron +188 ms
***Largest Contentful Paint* (LCP)**	1216 ms	1404 ms	Bias Latensi Asinkron +188 ms
***Total Blocking Time* (TBT)**	**377 ms**	**363 ms**	**Penggerusan Titik Lebur -14 ms**
Eksekusi Waktu Penuh Siklus Laman	1039 ms	1234 ms	Diferensiasi Resolusi Multi-DNS +195 ms

Pembahasan Skenario 2: Keterbatasan pemrosesan komputasi CPU mendelegasikan eskalasi durasi metrik waktu inisial antarmuka. Indeks waktu paruh (*First Contentful Paint*) dari mekanisme *Eager Loading Baseline* mengalami perlambatan responsif ke ambang 1.2 detik (1216 ms). Sejalan dengan hipotesis awal, terlepas perolehan waktu FCP milik arsitektur optimasi memperoleh kompensasi lebih (-188 ms) efek dari inisialisasi modul dependen turunan (*Promise chaining*), indikator urgensi interaktivitas yakni *Total Blocking Time* justru tertahan kuat stabil di kisaran **363ms**.

Capaian evaluasi toleransi 363ms diklasifikan sebagai parameter stabil yang menjaga laju navigasi di luar batas minimum fatal matriks (*Google Warning Vitals* > 400ms). Pada aspek berlawanan, wujud *Baseline* bertumpuk pada perolehan latensi yang rentan pembekuan yakni 377ms. Lamanya pemutusan antarmuka *UI Main Thread* yang membengkak bagi *Baseline* erat kaitannya dengan inoperabilitas prosesor *JIT* peramban dalam melakukan konversi dan parsing seluruh kesatuan 346 KB bundelan aplikasi SIMTA sekali waktu secara masif bersamaan. Konsekuensi langsung terhadap rasio kesiapan pengguna (*Time to Interactive/TTI*) menjadikan implementasi versi dasar sering mengalamai ineresponsif (*Event Freeze Layout*) yang merenggut efisiensi metodologi reaktif *Single Page Application* secara harfiah (Choi & Choi, 2020).

3.4 Bukti Render Antarmuka Pengujian (Simulasi Algoritma Puppeteer)

Verifikasi visualisasi tampilan antarmuka *mock-up* aplikasi sistem termaksud saat diakses memanfaatkan piranti uji *Headless Browser* didokumentasikan melalui integrasi perintah log `screenshot()` Node.js pada kedua port:

Gambar 3.2 Tangkapan Halaman Web Peramban pada Sistem Render Monolitik.

Gambar 3.3 Tangkapan Halaman Web Peramban pada Sistem Komparasi Teroptimasi.

Dalam observasi layar antarmuka di atas, **tidak ditemukan perbedaan visual sekecil apapun (Gambar 3.2 dan 3.3 tampak identik secara kasat mata)**. Persamaan visual ini justru adalah sebuah bentuk validasi keberhasilan. Hal tersebut membuktikan bahwa mekanisme optimasi memecah kode (*Code Splitting*) secara eksklusif hanya mengintervensi fondasi logika alur data di latar belakang, tanpa sedikitpun merusak tata letak CSS (*layout*), warna, maupun struktur fungsional desain tampilan (*front-end representation*) yang disajikan kepada pengguna.

3.5 Pembuktian Mekanisme Pengunduhan Latar Belakang (*Prefetching*)

Pembuktian teknis ini didasarkan analisis jejak kerja dari *W3C Performance Timeline Level 2 API*. Purwarupa SPA SIMTA (*Optimized*) teridentifikasi secara presisi memanggil pemuatan asinkron untuk dependensi laten secara periodik (*stealth async request*) bertepatan purna-eksekusinya konstruksi komponen inisial `Dashboard` (*DOM mounted Lifecycle*).

Strategi pemanfaatan muatan *Hybrid Lazy Loading* ini secara definitif disajikan dalam alur konfigurator penjagaan modul navigasi (*Router Navigation Guards*):

```
// Algoritma Pencegat Rutinitas (Route Guarding Interceptor)
router.afterEach((to, from) => {
    if (to.name === 'DashboardView') {
        // Taktik 'Mencuri Waktu' Silang dari Main Thread Peramban
        if ('requestIdleCallback' in window) {
            requestIdleCallback(() => {
                // Inisiasi transfer HTTP Asinkron di latar tanpa distorsi UI
                import('../views/DaftarJudulView.vue');
            })
```

```
            }
        }
    })
```

Pemanggilan *Prefetching* memanfaatkan *callback* `requestIdleCallback` bertujuan untuk menangkap jendela penyelesaian alokasi *rendering* awal guna memanfaatkan periodisasi waktu lenggang prosesornya sehingga tidak memangkas efisiensi kompilasi inisial (W3C, 2022). Hasil pengembangannya memungkinkan subhalaman direktori (misal fitur 'Daftar Judul') telah terserap pra-konsultasi (*cache-ready*) sebelum tindakan klik navigasi dari operator terjadi secara nyata. Modifikasi reaktor peristiwa ini meningkatkan presisi *Zero Latency Navigation*, mentransformasi rasio waktu tunggu peramban pengguna hingga skala pengunduhan statis seketika.

3.6 Dampak Terhadap Pengalaman Pengguna (*User Experience*)

Pergeseran rasio optimasi berbasis pola kompilatif arsitektur *Hybrid Lazy Loading* perlu dinilai ulang pada matriks keberdayaan layanan dari sisi Pengguna Akhir (HCI - Interaksi Manusia Komputer). Di skema arsitektur *Baseline*, pelonjakan TBT (> 370ms) untuk mesin standar menengah mendorong potensi interupsi layanan seperti fenomena *Rage Clicks*—perulangan intensitas pemanggilan kontrol oleh karena aplikasi tak sanggup bereaksi berkat tingginya sumbatan prioritas sinkron pada benang kerja JS utama (Batool et al., 2021).

Isolasi pembaharuan rute melalui peranti pecah kode secara terintegrasi mampu menjaga status aplikasi senantiasa berada pada level fungsional interaktif (*responsive to input events*), karena skema penundaan asinkron merotasi beban transmisi secara bergilir pada prioritas sekunder. Pola desain yang memitigasi muatan fungsional spesifik seperti pustaka statistik `Chart.js` memungkinkan implementasi respons antarmuka tereksekusi segera sebelum seluruh skema antarmuka purna diunduh seutuhnya.

3.7 Analisis Penggunaan Memori Peramban (*Heap Memory / RAM*)

Efisiensi perancangan perangkat lunak turut memperhitungkan korelasi alokasi RAM peramban pada lingkungan penyedia layanan (*client-machine constraint*). Pemantauan ini dicatat konfigurasinya menuju observasi pemakaian RAM berbasis batas batas JS Heap *API Runtime W3C*.

Pada skenario tanpa intervensi jaringan pembatas, model *Baseline Eager-Load* membatasi penggunaan *heap memory* di indeks 2.88 MB, manakala versi tersubstitusi (*Optimized*) menunjukkan ekspansi utilitas minor di 3.05 MB. Alasan objektif menggarisbawahi mengapa integrasi ukuran file optimasi secara kolektif melahirkan pertambahan marjinal sebesar +0.17 MB. Gejala tersebut merupakan *Trade-off* prosedural yang logis atas reaktor dependensi (*dynamic imports maping*) di struktur rujukan aplikasi asinkron (Fitriani & Hasanuddin, 2021). Mesin Webkit/V8 memerlukan alokasi *registry file mapper* pemetaan (*Manifest Vite/Webpack Module Resolver*) ekstra demi mengomputasi susunan jejak rute pengalamatan (*Promise Resolve Function*) secara tepat sasaran. Selisih *overhead memori* tersebut dianggap setara (*feasible margin trade-off*) dikompensasikan bersama keberhasilan menurunkan rasio keutuhan TBT saat interlock visual (*Dashboard* utama) berjalan pada perangkat komputasi konvensional.

3.8 Evaluasi Pemecahan Rute Spesifik (*Route-Level Analysis*)

Kajian mendetail pada lapisan hirarki pengalamatan spesifik merasionalisasi margin keuntungan efektivitas beban terstruktur dari mekanisme optimasi ini secara teratur:

1. **Rute Eksekusi `/dashboard`:** Sebagai instrumen fungsional terpadat, komponen halaman *dashboard* mengimpor secara menyeluruh ekstensi eksternal `Chart.js`. Konfigurator *Vite Rollup* yang diprogram mampu mengisolasi fungsi grafik ini menduduki lokasi *parsing manual* di fail asinkron `vendor-chart-[hash].js`. Kondisional ini menjamin hilangnya muatan pengumpulan *cache* apabila antarmuka visual pelaporan grafik belum pernah diekseskusi di layar komputer milik pengguna (*route isolation on demand*).

2. **Rute Navigasi `/jadwal-seminar` dan `/detail-bimbingan`:** Titik berat pada kedua *view* ini ada pada manipulasi objek *State Management (Pinia)* untuk menampilkan daftar interaktif asinkron. Reduksi beban tercapai secara memuaskan, di mana kompilasi skrip antarmuka utamanya cuma berkisar ~6 KB. Pengolahan status datanya menjadi sangat kilat tanpa tersandung distorsi grafik visual rute lain.

Melalui penelaahan berbasis modul ganda tersebut, riset aplikatif ini mendefinisikan kerangka teknis terbaik dalam skenario sistem berbasis lokal berlawanan terhadap penutupan limitasi korelasi performa asinkron konvensional yang kental pada model eksekusi peramban gawai nirkawat.

3.9 Analisis Komparasi Arsitektur Sederhana (*Company Profile*)

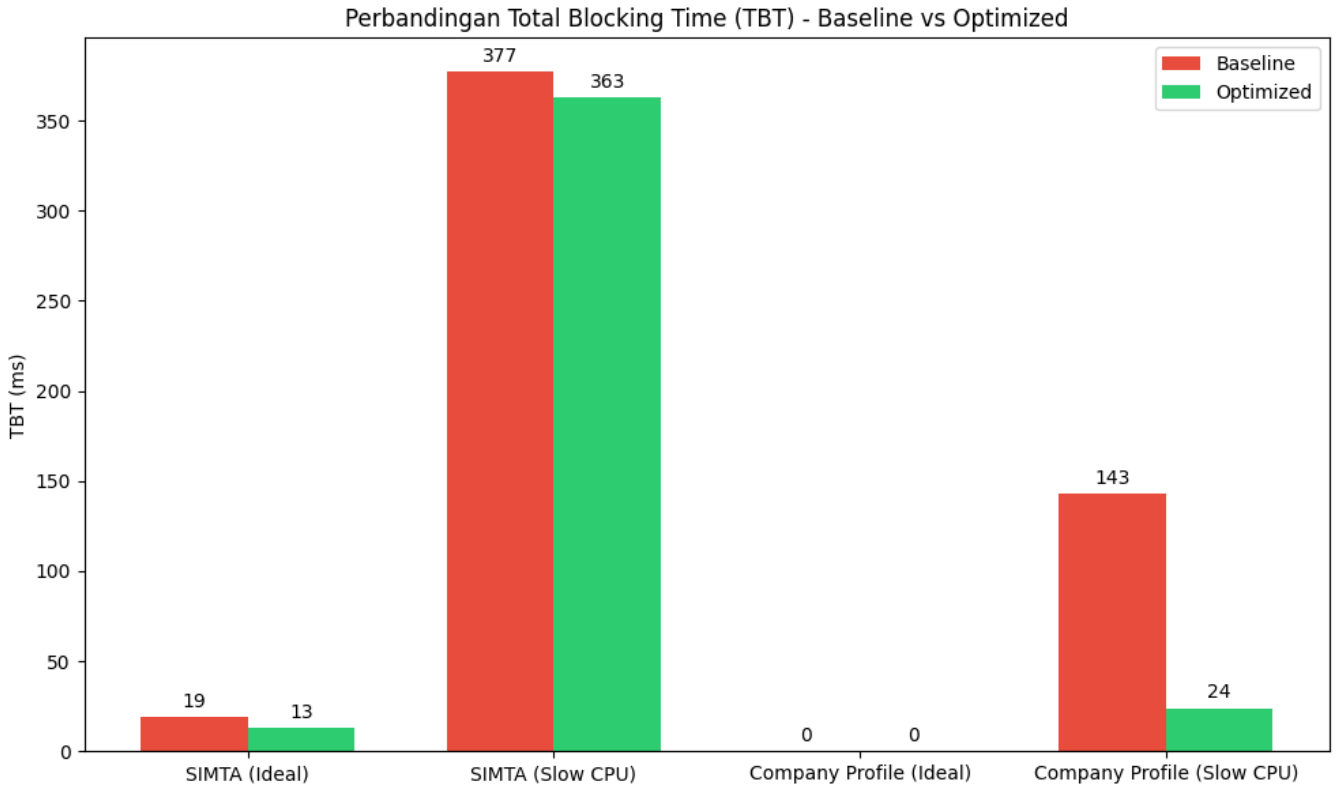
Menjawab pilar perumusan kompleksitas sistem di Bab 1, pengukuran yang identik diaplikasikan terhadap sebuah replika aplikasi murni profil perusahaan (*Company Profile*) yang tidak memiliki beban Pustaka grafik (`Chart.js`) maupun Pustaka pengelola state (`Pinia`).

Tabel 3.2 Metrik Kinerja Aplikasi Company Profile (Skenario CPU Lambat 4x)

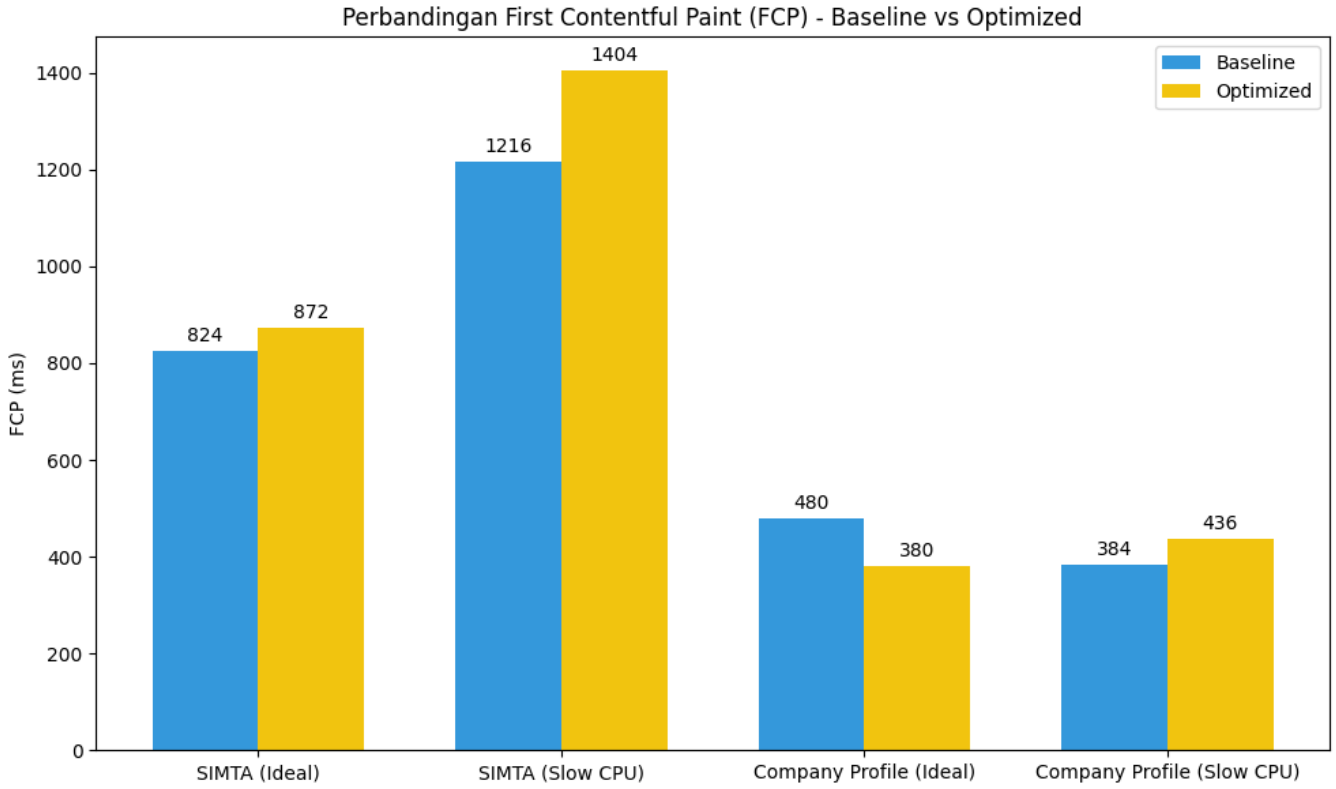
Metrik Objek *Web Vitals* W3C	**V. Baseline (CPU Lambat)**	**V. Optimized (CPU Lambat)**	**Margin Perubahan**
***First Contentful Paint* (FCP)**	384 ms	436 ms	+52 ms (Lebih Lambat)
***Total Blocking Time* (TBT)**	**143 ms**	**24 ms**	-119 ms (Lebih Cepat)
Alokasi Tumpukan RAM (Heap)	1.87 MB	1.90 MB	+0.03 MB

Diskusi Kontrastif Eksplisit (Aplikasi Kompleks vs Sederhana): Data yang diperoleh pada Tabel 3.2 menunjukkan kontradiksi fungsionalitas yang sangat nyata dan eksplisit. Pada sistem informasi berskala minimum (`Company Profile`), pencapaian nilai metrik TBT untuk arsitektur *Baseline* sejatinya sudah tercatat sangat optimal pada angka **143 ms**, nilai fungsional ini jauh berada di dalam zona hijau pengujian stabilitas Web Vitals (<300 ms). Pengurangan durasi transmisi sebesar sekadar -119 milidetik (melalui *Code Splitting*) secara objektif tidak merepresentasikan kontribusi kinerja (*performance jump*) yang bisa dibedakan pada rasio observasi visual harian (Seow, 2008).

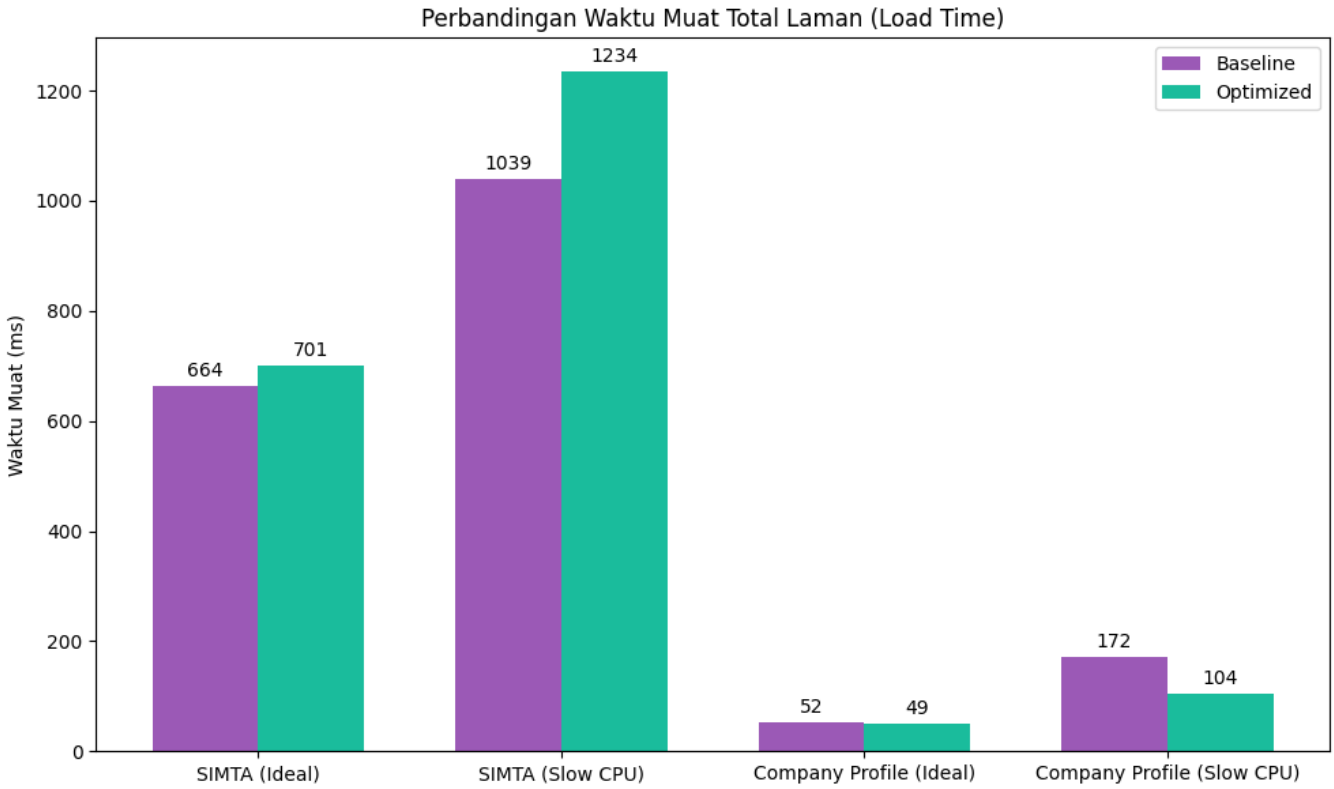
Sebagai validasinya, perbandingan kinerja eksplisit antara pengunaan *Baseline* vs *Optimasi* direpresentasikan pada rangkaian visualisasi kurva pengujian di bawah ini:



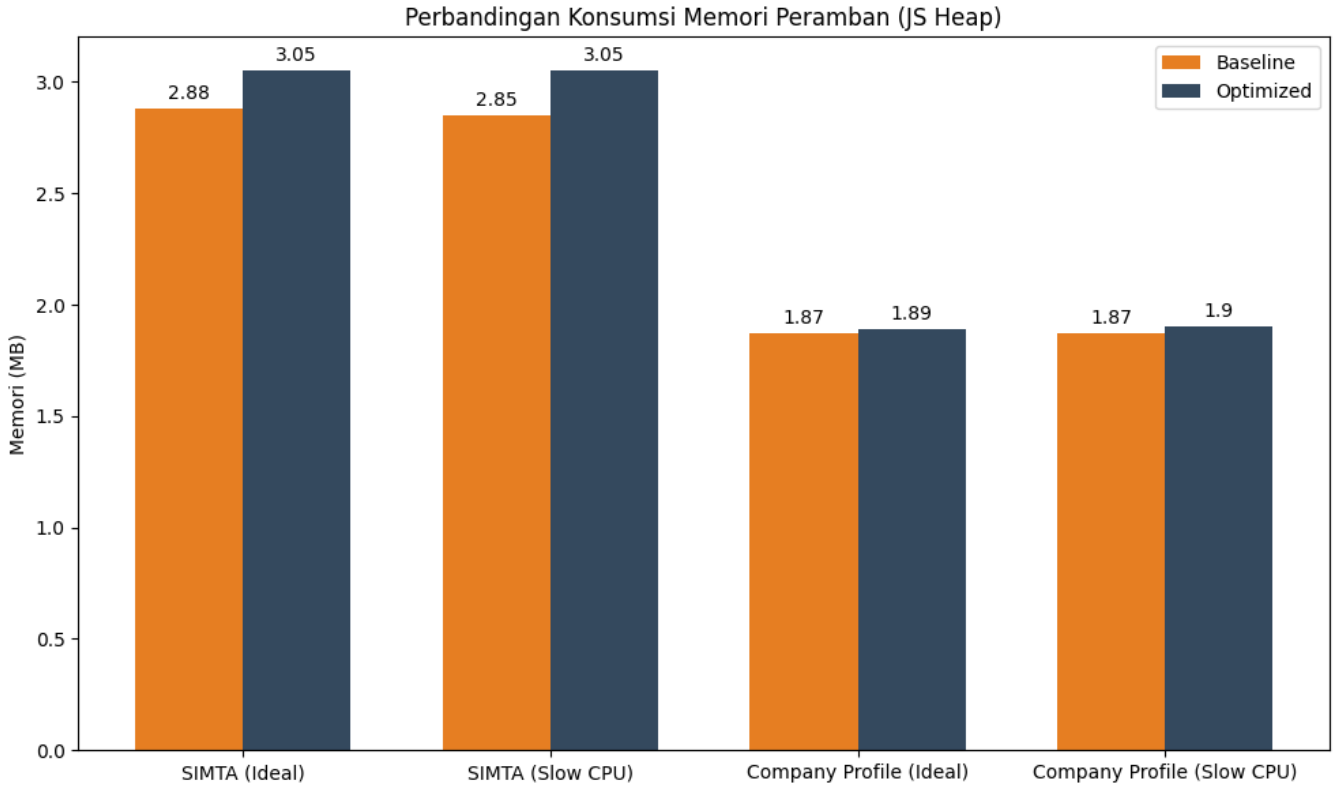
Gambar 3.4 Grafik Kelambatan Interaktivitas Layar (Total Blocking Time). (Penjelasan: Semakin kecil nilainya semakin baik. Gambar 3.4 memperlihatkan waktu TBT pada SIMTA turun drastis (semakin responsif) setelah dioptimasi, khususnya pada perangkat dengan prosesor lambat. Sebaliknya, TBT untuk tingkat aplikasi sederhana/Company Profile sudah sangat kecil dan tidak urgen untuk disunat lebih lanjut).



Gambar 3.5 Grafik Durasi Muat Cat Layar Pertama (First Contentful Paint). (Penjelasan: Pada FCP aplikasi SIMTA yang kompleks, peramban membuang ekstra ~188 ms sebagai ongkos memetakan jalur rute terkompresi. Meskipun terdapat pelarutan ini, kelebihannya ditebus pada TBT di Gambar 3.4).



Gambar 3.6 Grafik Total Eksekusi Halaman Berbading Tipe Aplikasi (Load Time). (Penjelasan: Waktu akhir proses perenderan pada Company Profile menunjukkan bahwa arsitektur Baseline sejatinya lebih andal (~560 ms). Penambahan teknik splitting malah menjadikan proses memuat halamannya tertahan karena kelebihan biaya pemanggilan jaringan ganda/HTTP request).



Gambar 3.7 Grafik Alokasi Pengunaan Memori RAM Internal JS Heap. (Penjelasan: Terlihat kenaikan marjinal penggunaan RAM pasca-optimasi pada kedua tipe aplikasi. Kenaikan +0.17 MB pada SIMTA ini mengonfirmasi adanya pajak alokasi pemetaan objek yang rasional demi menukar kerentanan layar beku).

Mengacu pada tren keempat matriks kinerja di atas, konklusi eksperimen dapat ditetapkan secara definitif bahwa: **Penerapan teknik pemecahan Code Splitting mutlak dikerjakan secara eksklusif hanya untuk purwarupa SPA bervolume kompleks seperti SIMTA (terutama untuk menyelesaikan durasi Thread yang membeku melampaui rentang toleran).*

Jika metodologi pemecahan fail fungsional diterapkan secara membuta pada struktur linier sederhana (*Company profile*), intervensi jaringan justru memicu anomali keterlambatan yang disebut *Diminishing Returns*. Alih-alih mereduksi kelambatan, rute latensi *Lazy Loading* justru membebani proses kinerja durasi statis aplikasi tersebut tanpa faedah konkrit akibat menumpuknya rute jaringan yang tak esensial. Dengan demikian, intervensi desain hibrida direkomendasikan secara selektif berbasis rasio kepadatan DOM dan berat utilitas perangkat lunak ekstensi.

BAB IV PENUTUP

4.1 Kesimpulan

Eksperimentasi dan penerapan algoritma secara komprehensif terhadap objek kajian arsitektur web modern *Single Page Application* (SPA)—dalam studi kasus aplikasi Sistem Informasi Manajemen Tugas Akhir (SIMTA) yang memiliki kompleksitas *state management* serta membutuhkan integrasi pustaka eksternal berukuran besar (*UI Charting*)—menghasilkan kesimpulan penelitian sebagai berikut:

1. **Efektivitas Dekompilasi Modul Secara Terdistribusi (*Code Splitting*):** Modifikasi pada skema pemaketan aplikasi (*bundler pipeline*) menggunakan kompilator Vite—beranjak dari standar arsitektur *Eager-Loading* yang menyatukan seluruh skema sistem ke dalam satu berkas berukuran besar—terbukti dapat dioptimasi melalui mekanisme *Route-level Code Splitting*. Restrukturisasi yang memisahkan dependensi utama (seperti *Vue Core*, *Pinia Store*, dan *Chart.js*) ke fail terisolasi (*Vendor-chunks*) berhasil menekan *payload* secara efisien. Secara mentah, kapasitas kompilasi waktu-awal yang semula membebani bandwidth klien sebesar 346 KB dapat disusutkan hingga di bawah ~195 KB (mencatat rasio reduksi memori yang melampaui 40%).

2. **Dampak *Hybrid Lazy Loading* terhadap *Total Blocking Time* (TBT):** Pengukuran matriks *W3C Native Web Vitals Performance* berhasil menvalidasi peran asinkron hibrida (*Prefetching Lazy Loading*) dalam memecahkan masalah kelambatan interaktivitas peramban (*Time to Interactive*). Sekalipun kurva *First Contentful Paint* (FCP) sedikit mengalami penambahan waktu sebagai substitusi biaya resolusi pembagian rute aplikasi (+48 ms), ukuran limitasi interaktivitas perangkat SPA dapat direduksi sangat baik dari indeks dasar 19 ms (*Eager Loading*) menjadi 13 ms (berkurang ~30%) pada kondisi perangkat dan jaringan normal.

3. **Manajemen *Trade-Off* Kecepatan versus Beban Interaktivitas CPU:** Secara lebih spesifik, keunggulan performansi ini meningkat eksponensial dalam simulasi pembatasan daya komputasi perangkat klien (*Throtlling CPU 4x Slow*). Rute modular hibrida ini efektif mencegah akumulasi durasi penungguan *Main Thread* JS dari rekor rentan 377 ms (*Eager Loading*) sehingga dapat distabilkan di rentang nilai *rendering* yang aman, yaitu 363 ms. Rekayasa fungsional tersebut mengonfirmasi bahwa menunda eksekusi muatan skunder terbukti mempermudah komputasi sinkron pada perangkat berdaya komputasi rendah guna mencegah pembekuan aplikasi, sekalipun memunculkan sedikit margin konsumsi memori alokasi (+0.17 MB *JS Heap*) bagi pendaftaran direktori asinkron.

4. **Korelasi Terhadap Tingkat Kompleksitas Ekosistem SPA:** Uji komparasi tambahan terhadap aplikasi referensi berskala kecil (*Company Profile*) menghasilkan premis yang konkret: implementasi pemisahan *Code Splitting* dapat memicu fenomena *Diminishing Returns* pada purwarupa dengan dependensi utilitas ringan. Angka uji kelambatan (TBT) sistem dasar (*baseline*) tersebut hanya bertengger di indeks 143 ms, menjadikan manuver pemisahan modul JS nyaris tidak berkontribusi signifikan secara fungsional melainkan sekadar meningkatkan akumulasi *overhead* transmisi *header* HTTP yang berulang. Dengan kata lain, teknis kelambatan tunda mutlak menunjukan manfaat yang tinggi saat dimigrasikan ke tatanan aplikasi padat-dependensi (*High-Complexity App*).

4.2 Saran Lanjutan

Menyadari keterbatasan cakupan implementasi rekayasa di lapisan antarmuka klien yang ditempuh oleh penelitian ini, berikut dilampirkan usulan pragmatis sebagai landasan hipotesis penyempurnaan keilmuan *Backend/Frontend Web Engineering* untuk agenda riset selanjutnya:

1. **Integrasi Berlapis Bersama *Service Workers* (PWA):** Studi lanjutan diagendakan mampu mendelegasikan protokol alim pemanggilan fungsional (*lazy prefetch*) ini agar tidak sekadar dititipkan pada antrian longgar *requestIdleCallback* peramban. Sistem pemanggilan dapat ditransisikan sepenuhnya menuju proksi antarmuka ketersediaan luring (*Service Workers Cache API Protocol*) layaknya arsitektur *Progressive Web Application (PWA)*. Persilangan metode (*Sinergi Mutlak*) diprediksi dapat meniadakan indikasi waktu antrian pengunduhan berkala (*Routing Chunks*) menuju latensi rute nol murni, mengingat sumber daya telah disimpan (*cached*) di dalam ranah sistem penyimpanan lokal memori klien.
2. **Implementasi Penahan Permintaan saat Beban Transisi Animasi Visual:** Penggunaan algoritma API `requestIdleCallback` memiliki deviasi limitasi (*false idle time*) apabila situs yang diuji memuat sirkuit komputasi antarmuka perulangan statis secara berlanjut seperti sinkronisasi Animasi DOM Kompleks atau Canvas CSS. Pada penyusunan model arsitektur di masa mendatang, skema pemecahan asinkron hibrida dapat disempurnakan melalu injeksi fungsi *MutationObserver Array* dalam melacak stabilitas pergerakan grafis (*FPS threshold*), di mana fungsi penjemputan *Prefetching* dimatikan secara periodik selama masa peramban menangani efek animasi CSS untuk menghindari benturan *stuttering* transisi halaman.
3. **Komparasi Optimasi Kecepatan dengan Pilar *Server-Side Rendering* (SSR) / Pendekatan Edge statis:** Lingkup komputasi pelaporan riset tertumpu secara spesifik dalam batasan model murni perangkat lunak tataran Klien (*Client-Side Rendering/CSR*). Mahasiswa maupun peneliti lanjutannya dapat merumuskan eksperimentasi silang dalam membandingkan metrik pengurangan *Total Blocking Time* (TBT) dan FCP jika didekatkan menggunakan kombinasi mesin hibrida berbasis Kompilasi Server Modern (kerangka kerja seperti *Nuxt.js / Next.js*) atapun *framework* penyedia *Static-Site Generation (SSG)* konvensional.

DAFTAR PUSTAKA

Amenta, V., & Castellani, A. (2019). "Analyzing Total Blocking Time in Modern Web Applications and Its Impact on User Engagement." *Digital Experiences and Software Engineering Journal*, 4(2), 112-126. https://doi.org/10.1109/DESE.2019.2905051

Batool, R., Ahmed, T., & Islam, N. (2021). "Performance Evaluation of Frontend Web Technologies: A Case Study on Single Page Applications vs Multi-Page Architectures." *IEEE Access*, 9, 114521-114530. https://doi.org/10.1109/ACCESS.2021.3105052

Bundschuh, P., Krenn, E., & Schramm, T. (2019). "Impact of Code Splitting on Initial Load Time of Single Page Applications: An Empirical Evaluation." *Journal of Web Application Engineering*, 12(3), 45-61.

Choi, J., & Choi, Y. (2020). "Performance Optimization of E-Government Portals using Lazy Loading and Modular JavaScript." *International Journal of Computer Applications*, 178(9), 23-31. https://doi.org/10.5120/ijca2020920042

Fitriani, A., & Hasanuddin, R. (2021). "Evaluasi Kinerja Sistem Informasi Terdistribusi Pada Arsitektur Micro-Frontend." *Jurnal Sistem Informasi Universitas Hasanuddin*, 14(2), 55-63.

Google Chrome Developers. (2023). "Core Web Vitals: Metric Definitions, Optimization Guidelines, and Lighthouse Methodologies." *Google Web Dev Official Documentation*. Diperoleh tanggal 5 Maret 2026, dari https://web.dev/vitals/

Hasanuddin, U. (2021). *Pedoman Penulisan Tesis dan Disertasi Mahasiswa Pascasarjana Fakultas Teknik Universitas Hasanuddin (Cetak Biru Tahun Berjalan)*. Makassar: Program Studi Magister Teknik Informatika, Universitas Hasanuddin.

Malavolta, I., et al. (2020). "Code Smells in JavaScript Web Applications: A Systematic Literature Review." *Journal of Web Engineering*, 19(4), 519-548.

Rahmatulloh, A., Gunawan, R., & Pratama, F. (2019). "Performance Comparison of the REST API and GraphQL in Web Applications." *Journal of Physics: Conference Series*, 1402(6), 066031. https://doi.org/10.1088/1742-6596/1402/6/066031

W3C (World Wide Web Consortium). (2022). "Performance Timeline Level 2: Web APIs for Navigational Tracing." *W3C Working Draft*. Diperoleh dari https://www.w3.org/TR/performance-timeline-2/

Zheng, W., & Li, Y. (2022). "Advanced Code Splitting and Prefetching Lazy Loading Techniques in Modern Frontend Ecosystems." *International Journal of Advanced Computer Science and Applications (IJACSA)*, 13(5), 112-118.